

D00 嵌入式工业平台

固件 OTA 升级系统 · 顶级说明书

从零精通：架构 · 协议 · 安全 · 实战 · 排障

STM32F407ZGT6 · FreeRTOS · 多通道传输 · 端到端安全

文档版本：V2.0（极致详解版） 编制：D00 工程团队

摘要

本文档是 D00 嵌入式工业平台的固件 OTA（Over-The-Air，空中升级）系统完整技术说明书，面向从零开始接触嵌入式升级体系的读者，力求“看完即精通”。全文共 14 章 + 3 个附录，以“背景知识 → 系统架构 → 物质基础（存储）→ 协议细节（包格式）→ 安全模型 → 传输通道 → 状态机 → 运行时序 → 可靠性 → 动手实操 → 实测证据 → 边界与排障 → FAQ → 展望”为主线，逐层展开一套生产级 OTA 系统的全部设计决策与实现细节。

系统以 STM32F407ZGT6 为核心，FreeRTOS 承载业务，LVGL 驱动人机界面；升级链路采用 BOOT/RUN 双区布局 + 外部 SPI Flash 双槽（下载暂存 ota_dl 与回滚源 img_lib），以 AES-256-CTR 加密、SHA-256 + ECDSA P-256 双公钥签名、芯片 UID 绑定、构建号防重放、版本号防回滚构建端到端安全链；支持 TCP / HTTP / 串口 HOSTLINK / CAN / YMODEM 救援五类通道；以 PENDING 启动确认、自动回滚、BACKUP 自愈、断点续传、断电四阶段恢复保证“任何一步失败都不变砖”。文中所有协议帧、数据结构、状态机、故障码均给出源码级定义，并附真实设备上的功能与安全实测记录，可直接作为开发、联调、维护与二次开发的一手依据。

关键词：OTA 升级；安全引导；AES-256-CTR；ECDSA；芯片绑定；防重放；防回滚；双区布局；外部 Flash 回滚源；断点续传；断电保护；状态机；STM32F407；FreeRTOS

目 录

摘 要.....	1
第 0 章 OTA 与嵌入式基础（新手必读）	1
0.1 什么是固件？什么是 OTA？	1
0.2 STM32F407 的 Flash 与启动机制	1
0.3 密码学基础（够用即可）	1
0.4 看门狗（Watchdog）	2
0.5 本系统 OTA 的一句话总结	2
第 1 章 系统总体架构	2
1.1 三端职责	2
1.2 一条升级消息的完整旅程（30 秒理解全系统）	3
1.3 设计原则	3
第 2 章 存储与分区架构（升级的物质基础）	3
2.1 为什么这样分区	3
2.2 内部 Flash 逐区详解	4
2.2.1 BOOT（0x08000000，64KB）	4
2.2.2 RUN（0x08010000，832KB）	4
2.2.3 PARAM（0x080E0000，128KB）	4
2.3 外部 Flash 逐区详解	5
2.3.1 ota_dl（0x000000，2MB，单槽 1MB）	5
2.3.2 img_lib（0x200000，头 4KB + 数据 832KB）	5
第 3 章 固件包格式（逐字节讲透）	5
3.1 包总体结构	5
3.2 字段逐个说明	5
3.3 打包过程（上位机视角，逐步）	6

3.4 解包过程 (BOOT 视角)	6
第 4 章 安全模型 (攻击者视角的攻防推演)	6
4.1 威胁与防线总览	6
4.2 信任根与密钥体系	7
4.3 校验顺序与失败策略	7
4.4 升级提交后的安全 (启动确认与回滚)	7
第 5 章 多通道传输架构 (五条路, 一个核心)	8
5.1 传输抽象层: 为什么这样设计	8
5.2 通道一: HOSTLINK (UART1, 921600)	8
5.3 通道二: ETH-TCP (:9020) ——字节级协议	8
5.4 通道三: ETH-HTTP (客户端拉取)	9
5.5 通道四: CAN (已实现, 1Mbps)	9
5.6 通道五: YMODEM (BOOT 直接接收——救援路径)	9
5.7 通道选型决策表	10
第 6 章 BOOT 启动与升级状态机 (逐状态讲透)	10
6.1 状态定义	10
6.2 启动状态机 (复位后的完整决策树)	10
6.3 升级模式内部流程 (boot_apply_download——逐步)	11
6.4 状态帧广播 (上位机可视化)	11
6.5 看门狗与长操作	11
第 7 章 运行时 OTA 全流程 (一次升级的完整时序)	12
7.1 阶段总览	12
7.2 阶段①下载 (Ota_Begin/Data/End 核心)	12
7.3 断点续传 (跨复位)	12
7.4 启动确认 (防回滚闭环)	12

7.5 一次典型 TCP 升级的完整时间线（实测 build 9180→9181）	12
第 8 章 可靠性设计（把“变砖”彻底排除）	13
8.1 断电保护矩阵（全生命周期）	13
8.2 看门狗体系（三层）	13
8.3 防呆与自愈清单	13
8.4 已知风险与规避（诚实记录）	14
第 9 章 上位机工具链（动手篇）	14
9.1 工具一览	14
9.2 快速上手：5 分钟完成一次 TCP 升级	14
9.3 OTA_Tool GUI 操作	14
9.4 打包与签名（encrypt_and_sign 流程）	15
9.5 版本管理约定	15
第 10 章 实测试验记录（全部真机）	15
10.1 功能实测矩阵	15
10.2 安全校验实测	15
10.3 编译与回归	16
第 11 章 已知边界与局限	16
第 12 章 故障排查指南（症状 → 原因 → 解决）	16
12.1 错误码速查表	16
12.2 常见故障排查步骤	17
12.2.1 症状 1：推送显示成功但固件没变	17
12.2.2 症状 2：设备不响应 :9020	17
12.2.3 症状 3：升级后启动黑屏/异常	17
12.2.4 症状 4：断点续传失败	17
12.3 诊断三板斧	18

第 13 章 FAQ 18

第 14 章 总结与展望 18

附录 A 关键数据结构（源码级） 19

 A.1 ota_header_t（32B） 19

 A.2 boot_param_t（PARAM 双份） 19

 A.3 OTA 会话槽（断点续传） 19

 A.4 TCP 帧..... 19

附录 B 术语表..... 19

附录 C 参考文档与源码索引 20

第 0 章 OTA 与嵌入式基础（新手必读）

本章为零基础读者铺垫理解本说明书所需的全部背景知识：什么是固件、什么是 OTA、STM32 的 Flash 与启动机制、以及密码学基础。已有经验者可直接跳到第 1 章。

0.1 什么是固件？什么是 OTA？

固件（Firmware）：烧录在微控制器（MCU）内部 Flash 中的机器码程序，是设备运行的“操作系统 + 应用程序”。本平台的固件即 APP（业务与界面）与 BOOT（引导与升级）两个镜像。

OTA（Over-The-Air）：“空中下载”，泛指不通过烧录器、而通过网络/总线把新固件传到设备并完成升级的方式。本系统的“空气”是 UART 串口、以太网（TCP/HTTP）、CAN 总线。

为什么不用烧录器：现场设备成千上万，逐一开盖接烧录器不现实；OTA 支持远程、批量、业务不中断地升级——但代价是必须把“烧录器做的事”（擦除/写入/校验）与“信任保障”（防篡改）全部在设备内软件化实现——这正是本说明书的主体。

※ 核心心智模型：OTA = 安全地把新固件从“传输通道”搬到“Flash 运行区”，并保证任何一步失败都能恢复。

0.2 STM32F407 的 Flash 与启动机制

内部 Flash：1MB，按扇区组织（0-3 号扇区 16KB、4 号 64KB、5-11 号 128KB）。擦除必须按扇区、写入按字/半字；Flash 只能 1→0 翻转，写前必须擦（擦后全 0xFF）。

启动流程：复位后 CPU 从 0x08000000 读取向量表（栈顶指针 + 复位向量）执行。因此 0x08000000 处必须是 BOOT 引导器；BOOT 校验 APP 后跳转 0x08010000。

外部 Flash：板载 SPI NOR Flash（如 W25Q128，16MB）独立于内部 Flash，本系统用它存放下载暂存（ota_dl）与回滚备份（img_lib）——即使升级中断电，数据不丢。

内部 Flash（1MB）			外部 SPI Flash（16MB）		
0x08000000	BOOT	64KB	0x000000	ota_dl	2MB（下载暂存，单槽 1MB）
0x08010000	RUN	832KB	0x200000	img_lib	头 4KB+数据 832KB（回滚源）
0x080E0000	PARAM	128KB	其余	日志/配置/用户数据	

0.3 密码学基础（够用即可）

对称加密（AES）：同一个密钥加密和解密。本系统用 AES-256-CTR（计数器模式）——流式处理，适合边解密边写入 Flash；密钥由每颗芯片唯一的 UID 派生，实现“包-设备绑定”。

非对称签名 (ECDSA)：私钥签名、公钥验签。上位机用私钥对固件包签名，BOOT 用内置公钥验证——保证包来自可信来源且未被篡改。椭圆曲线 P-256 是工业标准。

哈希 (SHA-256)：把任意长度数据压缩为 32 字节指纹；内容一变指纹即变。用于摘要+签名。

防重放：同一份合法固件包被重复提交（重放）应被拒绝——用“构建号”单调递增实现。

防回滚：攻击者用旧版本固件降级应被拒绝——用“版本号只升不降”实现。

※ 安全不靠“加密算法保密”，而靠“密钥/私钥保密”——算法全部公开（工业标准）。

0.4 看门狗 (Watchdog)

IWDG (独立看门狗)：由 LSI 低速时钟驱动，一旦启动不可关闭；软件必须周期性“喂狗”，超时即复位系统——防止程序卡死。本系统 BOOT 用 128 分频 (250Hz) + 4095 ≈16.4s（覆盖整段擦写），APP 用 32 分频 (1kHz) + 4095 ≈4.1s。

任务级看门狗 (WDOG)：APP 内软件层防线：监控各任务心跳，任务停滞超时即统一错误处理（转储+BKP 持久化+软复位）。

※ Flash 擦写是毫秒~秒级长操作，期间必须喂狗（BOOT 在 SRAM 中执行的 .ramfunc 代码里喂狗）。

0.5 本系统 OTA 的一句话总结

“多通道把加密签名固件包送进外部暂存区 → 复位进 BOOT → BOOT 校验（芯片/防重放/签名/版本） → 备份当前固件 → 擦除 RUN → 解密写入 → 置待确认 → 重启进新 APP → 新 APP 启动确认 → 闭环完成；任何一步失败都有对应自愈路径（续传/回滚/恢复模式）。”

第 1 章 系统总体架构

1.1 三端职责

端	角色	关键职责
BOOT	引导与升级执行者	启动校验、固件验证（芯片/防重放/签名/版本）、备份/擦除/解密写入、回滚与恢复、YMODEM 救援
APP	运行与升级服务者	业务运行、多通道接收固件、写入暂存、触发升级、启动确认、网络配置持久化
HOST	打包与调度者	加密签名打包、版本/build 管理、批量升级、阶段可

端	角色	关键职责
		视化、私钥管理

1.2 一条升级消息的完整旅程（30 秒理解全系统）



1.3 设计原则

- **通道与核心解耦**：同一套下载核心（Ota_Begin/Data/End），五条通道共享；新增通道仅注册一行（ota_transport 注册表）。
- **职责单一**：BOOT 只做“校验+执行”，体积最小（35KB）；业务全部在 APP。
- **可靠优先**：下载/备份/擦除/写入/提交每步都可断电恢复。
- **安全纵深**：加密+签名+芯片绑定+防重放+防回滚+密钥轮换纵深防御。

第 2 章 存储与分区架构（升级的物质基础）

2.1 为什么这样分区

Flash 升级的本质是“在新旧固件之间安全切换”。分区设计回答三个问题：①新固件先放哪（暂存区）；②万一失败退到哪（回滚源）；③升级状态记在哪（参数区）。

问题	答案（分区）	位置
新固件先放哪	外部 ota_d1 槽（下载暂存，单槽 1MB）	外部 Flash 0x000000
失败退到哪	外部 img_lib 槽（回滚源，备份上一版 RUN）	外部 Flash 0x200000
状态记在哪	PARAM 参数区（boot_state/count/build_no/会话槽）	内部 Flash 0x080E0000

问题	答案 (分区)	位置
新固件跑在哪	内部 RUN 区 (832KB, 方案 B 扩容)	内部 Flash 0x08010000

※ 方案 B 的来历：原设计在内部 Flash 划 BACKUP/DOWNLOAD 区，导致 RUN 仅 320KB；方案 B 把回滚源与下载暂存全部外移到外部 Flash，内部仅留 BOOT/RUN/PARAM，RUN 扩至 832KB。

2.2 内部 Flash 逐区详解

2.2.1 BOOT (0x08000000, 64KB)

- **内容：**向量表、启动状态机、升级执行（校验/备份/擦除/解密写/回滚）、YMODEM 接收、状态帧广播。
- **信任根：**固化 ECDSA 公钥（新 + LEGACY 双公钥，支持密钥轮换过渡期）。

2.2.2 RUN (0x08010000, 832KB)

- **页脚约定：**APP 向量表首地址；尾部 0x080DFFF8 写魔数 0x4F54412E、0x080DFFFC 写版本号——BOOT 跳转前以此判定“APP 是否有效”。
- **链接保证：**魔数/版本由链接脚本独立 load region (.ota_footer) 保证位于固定地址。

2.2.3 PARAM (0x080E0000, 128KB)

子区	地址	用途
boot_param slot0	0x080E0000	启动状态/计数/回滚计数/错误码/防重放 build_no
boot_param slot1	0x080E0400	双份冗余（间距 1KB），CRC-32 校验选优
网络配置 NVM	0x080E1000 (4KB)	APP 的 net ip 持久化（32B 槽 ×128 追加写）
OTA 会话槽	0x080E2000	断点续传：magic 0x4F54414D + version + total + received + crc32

- **冗余策略：**boot_param 双份 + CRC（CRC 只覆盖 crc32 字段之前，避免自引用）——单份损坏可自愈。
- **提交清理：**BOOT 升级提交后失效全部会话槽，防止旧包续传。

2.3 外部 Flash 逐区详解

2.3.1 ota_dl (0x000000, 2MB, 单槽 1MB)

- **下载暂存:** 运行时 OTA (TCP/HTTP/HOSTLINK) 分块写入; YMODEM 兜底同样收包到该槽。
- **预下载直通:** BOOT 探测包头 (magic 0x4F5441FE 且大小合理) → 直通应用, 无需重新下载。

2.3.2 img_lib (0x200000, 头 4KB + 数据 832KB)

- **回滚源:** 升级前将当前 RUN 全量备份到 img_lib; PENDING 回滚/自愈由 BOOT 从该槽恢复。
- **保留策略:** 新固件启动确认后备份仍保留至确认点——PENDING 期间回滚源始终可用。

第 3 章 固件包格式 (逐字节讲透)

3.1 包总体结构

ota_header_t (32 字节, 明文)				
magic	4B	0x4F5441FE ('OTA' 魔数)		
version	4B	固件版本 (如 213) ——防回滚比较		
firmware_size	4B	原始固件大小 (明文长度) ——容量校验		
aes_iv	12B	AES-CTR 初始向量 (+4B 零填充成 16B)		
chip_id	4B	目标芯片 ID (=DBGMCU->IDCODE & 0xFFF)		
build_no	4B	构建号 (如 9181) ——防重放比较		
载荷区 (密文): AES-256-CTR 加密的 APP 二进制 (长度=明文长)				
签名区 (64B): ECDSA P-256 签名 (对 头部+载荷 的 SHA256 摘要)				

3.2 字段逐个说明

字段	字节	含义	谁生成	谁校验
magic	4	包类型标识 (OTA 包 vs 其他)	上位机	BOOT 探测
version	4	固件版本号——只升不降 (防回滚)	version.json	BOOT (Rollback denied)
firmware_size	4	原始固件大小——校验 ≤832KB (RUN 容量)	上位机	BOOT
aes_iv	12	CTR 模式初始向量 (随机/可预测均可, CTR 不重复即可)	上位机	BOOT 解密用

字段	字节	含义	谁生成	谁校验
chip_id	4	目标芯片 ID——防跨芯片烧录	上位机（目标 UID）	BOOT（Chip mismatch）
build_no	4	构建号——严格递增（防重放）	version.json（递增）	BOOT（Replay denied）

※ 防重放实战（本平台真实案例）：PARAM 持久化 last_build_no；同构建号包（build=9180=last）被 BOOT 拒绝“Replay denied”；构建号递增（9181）后立即通过。

3.3 打包过程（上位机视角，逐步）

- 1. 读 APP.bin 原始固件
- 2. 构造 ota_header_t: magic/version(213)/size/aes_iv(12B)/chip_id/build_no(9181)
- 3. derive_aes_key: 由设备 UID 派生 32B AES 密钥（与 BOOT 一致）
- 4. AES-256-CTR 加密载荷（iv = aes_iv + 4B 零）
- 5. SHA256(头部 + 密文) 得 32B 摘要
- 6. ECDSA P-256 私钥（环境变量 OTA_PRIVKEY）对摘要签名 → 64B 签名
- 7. 输出：头部 + 密文 + 签名（encrypt_and_sign 完成）

3.4 解包过程（BOOT 视角）

- 1. 读外部 ota_dl 头部 → magic 校验
- 2. chip_id 校验 → build_no 防重放 → body_len 容量校验
- 3. SHA256(头部+密文) → ECDSA 双公钥验签（新/LEGACY 任一通过）
- 4. version 防回滚（>= 当前版本）
- 5. derive_aes_key (UID) → AES-CTR 流式解密，边解密边写 RUN
- 6. 写魔数/版本 → PARAM 置 PENDING → 重启

第 4 章 安全模型（攻击者视角的攻防推演）

4.1 威胁与防线总览

攻击手段	攻击者目标	防线	失败输出
窃听传输	拿到固件逆向	AES-256-CTR 加密（密钥=设备 UID，跨设备无意义）	密文无明文价值
篡改包体	注入恶意代码	SHA256+ECDSA 签名（私钥不在设备/仓库）	ECDSA verify failed!
伪造整个包	分发恶意固件	签名私钥不可得；芯片 ID 绑定	Chip mismatch!
重放旧包	回退到有漏洞版本	build_no 严格递增（PARAM 持久化）	Replay denied!

攻击手段	攻击者目标	防线	失败输出
降级回滚	利用旧版漏洞	version 只升不降 + 密钥轮换（旧包可能旧密钥）	Rollback denied!
跨设备烧录	一个包刷全部	chip_id == DBGMCU->IDCODE & 0xFFF	Chip mismatch!
读 Flash 提取	拿固件/密钥	RDP Level 1 读保护（发布前启用，DAP 不可读）	调试器无法连接

4.2 信任根与密钥体系

- **信任根**：BOOT 固化 ECDSA 公钥——设备内唯一信任锚；一切校验从公钥出发。
- **设备绑定**：AES 密钥 = f(芯片 UID)——包与设备绑定，无法跨设备复用。
- **密钥轮换**：BOOT 固化双公钥（新 + LEGACY）——密钥轮换过渡期新旧包都能验签；成熟后移除旧公钥。
- **私钥管理**：签名私钥仅存在于上位机环境变量（OTA_PRIVKEY），不入仓库、不出内存。

4.3 校验顺序与失败策略

BOOT 按“成本递增、尽早拒绝”顺序校验：芯片 ID（最快）→ 防重放 → 长度 → 签名（最贵）→ 版本。任一失败：打印原因、归一化参数（清除升级残留）、状态帧广播错误码、跳回 APP（支持重新下载/续传）——绝不卡死在升级模式。

4.4 升级提交后的安全（启动确认与回滚）

- **待确认**：新固件写入后置 PENDING；BOOT 每次启动 count++ 并跳 APP。
- **确认**：APP 首次正常运行调 ota_confirm_startup → PENDING 转 NORMAL（防回滚闭环）。
- **回滚**：新固件崩溃/未确认 → BOOT 计数到 3 → 自动回滚 img_lib 备份。
- **恢复**：回滚超限（rollback_count >= MAX，上限 5）→ RECOVERY 恢复模式；无有效包时自动归一 NORMAL 回旧固件（有意妥协：宁用旧固件不砖机），升级通道随时可恢复。

第5章 多通道传输架构（五条路，一个核心）

5.1 传输抽象层：为什么这样设计

OTA 的下载核心（Ota_Begin/Data/End——擦暂存区、写块、断点续传、触发升级）与具体传输通道完全解耦。通道只做两件事：收字节、把字节按帧喂给核心。新增一条通道（如 CAN）只需实现服务并调用 OtaMgr_Register 注册一行。

```
typedef enum {
    OTA_TRANSPORT_UART    = 1, /* HOSTLINK 串口（data_link 通道） */
    OTA_TRANSPORT_ETH_TCP = 2, /* 以太网 TCP 服务器（:9020） */
    OTA_TRANSPORT_ETH_HTTP = 3, /* 以太网 HTTP 客户端（拉取） */
    OTA_TRANSPORT_CAN     = 4, /* CAN 总线 OTA（ota_can_svc 运行时注册） */
} ota_transport_id_t;

/* OtaMgr_Init 登记：UART / ETH-TCP / ETH-HTTP;
 * CAN 由 OtaCanSvc_Init 运行时注册（available=1）;
 * 注册表容量 OTA_TRANSPORT_MAX=6; ota status 命令逐条展示 */
```

5.2 通道一：HOSTLINK（UART1，921600）

- **载体**：上位机数据链路协议（data_link）承载；帧 = AA 55 + cmd + len + payload + CRC16-LE；OTA 命令号：0x08 BEGIN / 0x09 DATA / 0x0A END / 0x0B STATUS / 0x0D RESET（0x0C 为 BOOT 状态广播帧）。
- **会话**：载荷字节序为小端（LE）：BEGIN(version+size 各 4B LE) → DATA(offset 4B LE + ≤240B 块) → STATUS 查询 → END。
- **适用**：默认推荐；开发调试与有线现场升级。

5.3 通道二：ETH-TCP（:9020）——字节级协议

APP 内 ota_tcp_svc 作为 TCP 服务器监听 9020；ota_tcp_cli 作为客户端推送。帧格式：

通用帧 = magic(0x5A,1B) | cmd(1B) | len(2B 大端) | payload | crc8(1B)

cmd 定义：

```
0x01 BEGIN    payload: version(4B) + size(4B)（均大端；无 build 字段）
0x02 DATA    payload: offset(4B 大端) + 240B 固件块
0x03 END      payload: 空（触发升级）
0x04 STATUS   payload: 空（服务端回 status+已收/总长）
0x05 RESET    payload: 空（触发复位）
```

CRC8 = poly 0x07 初值 0，覆盖 cmd+len+payload；ACK 帧 cmd=0x80

DATA 帧长度上限 OTA_TCP_MAX_FRAME = 4 头+4 偏移+240 数据+1CRC = 249B

阶段	发起方	帧	接收方动作
握手	客户端	BEGIN(version,size)	擦外部 ota_d1 / 恢复同版本续传

阶段	发起方	帧	接收方动作
			会话；回 ACK
传输	客户端	DATA×N（每块 240B）	Ota_Data 写槽；每帧回 ACK(received)
查询	任一方	STATUS	返回已收字节（断点确认）
完成	客户端	END	校验完整性 → 写 PARAM UPGRADE + BKP 标志 → 复位
遥控	客户端	RESET	立即复位（救援场景）

※ 实战：ota_tcp_push.ps1 封装（自动读 version.json 的 version/build，调用 ota_tcp_cli 推送）。注意：build 必须比 PARAM last_build_no 大，否则被防重放拒绝——这是 OTA 失败的第一排查点。

5.4 通道三：ETH-HTTP（客户端拉取）

- **模式：**APP 作为 HTTP 客户端向服务器拉取固件包（ota_http_svc）。
- **适用：**适合服务器统一分发（固定 URL + 版本查询）。

5.5 通道四：CAN（已实现，1Mbps）

- **现状：**ota_can_svc 完整实现（CAN 帧 ↔ Ota_Begin/Data/End 协议翻译），运行时注册（available=1）。
- **帧 ID：**控制帧 0x200（主机→设备）：BEGIN/END/STATUS/ABORT；数据帧 0x201（240B 块，行帧规约）；应答 0x210、块 ACK 0x211（设备→主机，逐块背压，携带已收字节数）。
- **规约：**BEGIN 载荷：size(LE32) + version(LE16)；5s 无数据超时自动 ABORT；乱序/超长整组丢弃。
- **适用：**车载/工业总线无网络环境的首选；BSP CAN1 1Mbps 就绪即用。

5.6 通道五：YMODEM（BOOT 直接接收——救援路径）

- **触发：**触发：BKP 标志 / PARAM UPGRADE → BOOT 进升级模式；若 ota_d1 无有效预下载包 → 进入 YMODEM 等待。
- **流程：**BOOT 经 UART1（115200）接收 YMODEM 包，按 4KB 扇区边擦边写外部 ota_d1 槽 → 同一套校验/备份/解密/写流程。
- **定位：**APP 无法启动时的最后手段（配合 BOOT0 + UART 恢复）。

5.7 通道选型决策表

场景	推荐通道	原因
开发调试	HOSTLINK	921600 高速、双向可视
现场有线	HOSTLINK / ETH-TCP	稳定可控
远程在线	ETH-TCP / ETH-HTTP	网络可达即可
总线环境	CAN (1Mbps)	无网络依赖
APP 崩溃	YMODEM (BOOT 直收)	不依赖 APP

第6章 BOOT 启动与升级状态机（逐状态讲透）

6.1 状态定义

状态	值	含义
NORMAL	1	正常运行态：BOOT 校验魔数 → 跳 APP
PENDING	2	升级待确认：新固件已写，等待 APP 启动确认（防回滚窗口）
RECOVERY	3	恢复模式：回滚超限；无有效包时自动归一 NORMAL 回旧固件（有意妥协：宁用旧固件不砖机，升级通道随时可恢复）
UPGRADE	4	升级模式：执行升级流程

6.2 启动状态机（复位后的完整决策树）

复位

```

├─ BKP 标志=UPGRADE 或 param.state=UPGRADE(4) ──> 升级模式
├─ param.state=PENDING(2) 且 count>=3 ──────────> 回滚 (BACKUP→RUN)
├─ param.state=PENDING 且 count<3 ────────────> 待确认 (count++ → 跳 APP)
├─ param.state=RECOVERY(3) ───────────────────> 恢复模式 (等固件; 无有效包
                                                    自动归一 NORMAL 回旧固件)
└─ param.state=NORMAL(1)
    ├─ RUN 魔数有效 ──> 跳 APP
    ├─ RUN 无效且 BACKUP 有效 ──> BACKUP→RUN 复制 → 跳 APP (自愈)
    └─ 均无效 ──> 升级模式

```

※ 自愈路径：即使 RUN 被完全破坏，只要外部 img_lib 备份有效，BOOT 会自动恢复——这是“断电/坏写也不变砖”的根本保障。

6.3 升级模式内部流程（boot_apply_download——逐步）

步骤	动作	失败处理	错误码
1 探测	检查外部 ota_dl 包头（magic 0x4F5441FE 且总长 ≤1MB）→ 直通应用；无包回退 YMODEM	无包不卡升级模式	—
2 校验	芯片 ID → 防重放(build_no>last) → 容量 (firmware_size≤832KB) → SHA256+ECDSA(双公钥) → 版本防回滚	归一化参数，跳回 APP 支持重下/续传	SEC err
3 备份	RUN 有效则全量备份到外部 img_lib（写后读回逐块校验）	备份失败中止升级（RUN 未破坏）	0x1002
4 擦除	擦除 RUN 区（扇区 4-10）	RUN 已破坏，复位后 BACKUP 自愈	0x1003
5 写入	AES-CTR 流式解密写入 RUN，随后校验向量表（SP/PC）	RUN 损坏，复位后 BACKUP 修复 / 向量非法	0x1004/0x1005
6 提交	写魔数/版本 → PARAM 置 PENDING+count=1 → 清会话槽 +擦外部下载槽（备份保留）→ 状态帧广播 → 重启	PENDING 持久化（断电安全）	—

6.4 状态帧广播（上位机可视化）

- **机制：**BOOT 应用阶段经 UART1 广播 0x0C 帧（阶段+错误码+版本，临时切 921600）。
- **阶段：**上位机解析 → 阶段流程条显示真实推进：VERIFY → BACKUP → ERASE → WRITE → COMMIT → DONE。

6.5 看门狗与长操作

- **窗口：**BOOT IWDG 128 分频（250Hz）+ 4095 ≈16.4s——覆盖 832KB 内部擦写 + 外部备份长操作。
- **喂狗：**Flash 擦/写/复制长操作在 SRAM 内执行（.ramfunc）并逐块喂狗——防止长操作触发看门狗复位。

第 7 章 运行时 OTA 全流程（一次升级的完整时序）

7.1 阶段总览

阶段① 下载（业务不中断）	APP 运行中，多通道收包写外部 ota_dl
阶段② 提交	END → PARAM UPGRADE + BKP 标志 → 复位
阶段③ BOOT 应用	探测→校验→备份→擦除→解密写→PENDING→重启
阶段④ 启动确认	APP 首启 → ota_confirm_startup → NORMAL

7.2 阶段①下载（Ota_Begin/Data/End 核心）

接口	入参	行为
Ota_Begin	version, size	若已在接收则拒绝；擦外部 ota_dl（或恢复同版本会话）；置 OTA_ST_RECEIVING
Ota_Data	offset, chunk	写外部槽对应偏移；每 16 块（3840B）持久化一次进度到会话槽（断点粒度 3840B）
Ota_End	—	校验接收完整 → 写参数 UPGRADE + BKP 标志 → BSP_SystemReset

※ 业务不中断的关键：下载期间 APP 一切照常（网络/界面/外设），仅 Flash 写入短暂占用。

7.3 断点续传（跨复位）

- **机制**：会话槽记录 version/total/received；BEGIN 检测同版本会话 → 不擦除，从 received 偏移续传。
- **恢复**：恢复路径重置 Flash 控制器状态（清错误标志 + Unlock/Lock），避免续传写入 BSY 卡死。
- **实证**：实测：32160/64192 字节断点续传成功。

7.4 启动确认（防回滚闭环）

APP 启动 → ota_confirm_startup():

读 PARAM boot_param

若 boot_state == PENDING:

boot_state = NORMAL /* 新固件正常运行，确认成功 */

boot_param_save() /* 双份写 */

擦除外部 img_lib 备份头（4KB）→ 回滚源失效，杜绝旧备份复活

否则：无操作（正常运行态）

效果：确认前回滚源可用；确认后正式切换（A/B 闭环完成）

7.5 一次典型 TCP 升级的完整时间线（实测 build 9180→9181）

```
[00:00.0] ota_tcp_cli 连接 192.168.10.10:9020
[00:00.2] BEGIN v213 size=459204 build=9181
[00:00.5] DATA x1914 (1913x240B + 尾块 84B, 带 offset+CRC8)
```

```
[00:04.1] STATUS 459204/459204 → state=1 (完整)
[00:04.2] END → APP 写 UPGRADE → 复位
[00:05.0] BOOT: 防重放校验 build=9181>last=9180 通过 → 备份/擦除/解密写
[00:35.0] PENDING → 重启 → 跳新 APP → 启动确认 → NORMAL
[00:36.0] 升级完成, 业务恢复
```

第 8 章 可靠性设计（把“变砖”彻底排除）

8.1 断电保护矩阵（全生命周期）

断电时刻	现场状态	上电恢复路径	验证
下载中	外部 ota_dl 部分写入	会话槽续传（同版本）	✅ 实测
备份后	img_lib 已备份, RUN 未动	重启重新应用（RUN 仍有效）	✅ 实测
擦除后	RUN 全 0xFF	BOOT 探测 ota_dl → 重新应用	✅ 实测
写入后	RUN 部分写入（无魔数）	重新应用 / BACKUP 回滚	✅ 实测
提交后	PENDING 持久化	启动确认（count++ 直到确认或回滚）	✅ 实测

※ 设计哲学：任何时刻的“系统状态”要么已持久化（PARAM/外部 Flash），要么可重建（重下/重备份）——不存在“中间态不可恢复”的窗口。

8.2 看门狗体系（三层）

层	实现	窗口	防什么
BOOT IWDG	128 分频（250Hz）+4095 + SRAM 内喂狗	≈16.4s	BOOT 长操作卡死
APP IWDG	32 分频 + SysTick 钩子喂狗	≈4.1s	APP 卡死/任务饿死
任务级 WDOG	SysMon 监控各任务心跳，超时统一错误管理	5s（可配）	单任务停滞

※ 调试/取证模式（APP_DEBUG_MODE=1）：关闭 IWDG/WDOG/ERR 软复位——死机现场保留供 DAP 取证。

8.3 防呆与自愈清单

- **防呆**：芯片绑定、版本/构建号校验、参数区 CRC、双份冗余。
- **自愈**：校验失败归一化参数跳回 APP；RUN 损坏由 BACKUP 自愈；均无效进升级模式。
- **清理**：BOOT 升级提交后失效全部会话槽（防旧包续传）。

8.4 已知风险与规避（诚实记录）

- **UID 依赖：**批量升级要求同批次设备 UID 一致（AES 密钥由 UID 派生）。
- **回滚深度：**BACKUP 仅保留上一版（无 N-2 级回滚）。
- **BSY 卡死：**启动早期擦参数扇区 BSY 卡死为“证据充分的推测”，已通过架构规避（避免启动早期擦 PARAM）。

第 9 章 上位机工具链（动手篇）

9.1 工具一览

工具	形态	用途
OTA_Tool	GUI（Python）	版本库管理、串口升级、批量升级、阶段可视化
ota_tcp_cli.py	命令行	以太网 TCP 推送（脚本化/CI）
ota_tcp_push.ps1	PowerShell 封装	自动读 version.json，一行推送
gen_keys.py	命令行	生成 ECDSA 密钥对（私钥注入环境变量）
enable_rdp.py	命令行	启用 RDP 读保护脚本（发布前）

9.2 快速上手：5 分钟完成一次 TCP 升级

① 改版本：编辑 config/version.json

```
{ "ota_version": 213, "ota_build": 9181 }
```

 ← build 必须比设备 PARAM last_build_no 大

② 构建固件：Keil UV4 -b（或 CI）

③ 推送：

```
powershell -File workflow\ota_tcp_push.ps1 -Ip 192.168.10.10
```

④ 观察：下载进度 → END 触发 BOOT → 约 30s 完成 → 设备重启进新固件

⑤ 验证：主界面固件卡显示 FW v213 b9181（PARAM last_build_no 同步更新）

※ 最容易踩的坑：build 号没递增 → BOOT 防重放拒绝（Replay denied）→ 看起来“OTA 成功”但固件没变。每次推送前务必递增 ota_build。

9.3 OTA_Tool GUI 操作

- ① **版本库：**版本库刷新 → 选择固件版本（自动填文件/版本号/build）。
- ② **连接：**串口 COM / 921600；私钥环境变量 OTA_PRIVKEY 注入。
- ③ **升级：**打开串口 → 开始升级 → 阶段流程条 + BOOT 状态帧彩色日志。

- **④ 批量：**端口逗号分隔（COM5,COM13），串行预分配 build_no 后并发升级。

9.4 打包与签名（encrypt_and_sign 流程）

```
encrypt_and_sign(APP.bin, version, build, chip_id):
    hdr = ota_header_t(magic=0x4F5441FE, version, firmware_size, iv, chip_id, build_no)
    key = derive_aes_key_from_uid(uid)           # 与 BOOT 一致
    cipher = AES_CTR_encrypt(key, iv, payload)   # 载荷加密（长度=明文长度）
    digest = SHA256(hdr + cipher)
    sig = ECDSA_sign(privkey, digest)           # 私钥=环境变量 OTA_PRIVKEY
    return hdr + cipher + sig
```

9.5 版本管理约定

- **版本：**version 语义：功能版本，只升不降（防回滚）。
- **构建号：**build 语义：每次出包必须递增的流水号（防重放）。
- **单一事实源：**config/version.json 为单一事实源（CI/推送脚本自动读取）。

第 10 章 实测验证记录（全部真机）

10.1 功能实测矩阵

项目	结果
HOSTLINK 运行时升级	v13→v69 连续多轮成功（业务不中断）
ETH-TCP 升级	v213 build 9180→9187 连续 8 轮成功
YMODEM 传统升级	成功（含新包头兼容）
回滚（PENDING 超限）	[RB] erase=1 copy=1 成功
BACKUP 自愈（RUN 损坏）	自动恢复成功
断点续传	32160/64192 续传成功
断电注入 4 阶段	全部自动恢复
密钥轮换	新/旧公钥包均验证通过

10.2 安全校验实测

场景	BOOT 输出	结论
同构建号重放	[SEC] Replay denied! build=9180 last=9180	防重放生效（真实

场景	BOOT 输出	结论
		案例)
跨芯片	[SEC] Chip mismatch! pkg=0x%04X dev=0x%04X	芯片绑定生效
版本降级	[SEC] Rollback denied! New:%lu, Current:%lu	防回滚生效
签名损坏	[SEC] ECDSA verify failed!	验签生效

10.3 编译与回归

- **构建：**BOOT/APP Keil AC5 构建 0 Error / 0 Warning。
- **CI：**CI (GCC 构建 + ctest 主机测试 + HOST 测试 + 后门扫描) 全绿。

第 11 章 已知边界与局限

项	现状	影响/计划
RDP 读保护	脚本就绪，发布前实机验证	启用后 DAP 无法读 Flash（防提取）
启动早期擦 PARAM	寄存器级机制为推测，已架构规避	避免启动早期擦参数扇区
批量 UID 一致	AES 密钥由 UID 派生	同批次设备 UID 必须一致
回滚深度	BACKUP 仅上一版	无 N-2 级回滚
CAN 通道	已实现（1Mbps，ID 0x200/0x201），BSP 就绪即用	总线场景即可启用
HTTPS	HTTP 拉取（明文传输）	包已加密签名，传输层明文可接受
img_lib 容量	备份槽 896KB（头 4KB + RUN 832KB + 余量 60KB）	与 RUN 上限匹配

第 12 章 故障排查指南（症状 → 原因 → 解决）

12.1 错误码速查表

错误码/输出	含义	处理
Replay denied!	build_no 未递增 (<= PARAM last_build_no)	递增 ota_build 后重推

错误码/输出	含义	处理
Chip mismatch!	包 chip_id 与设备不符	用目标设备 UID 重新打包
ECDSA verify failed!	签名错误/私钥不符/包被篡改	检查 OTA_PRIVKEY 与打包流程
Rollback denied!	version 低于当前固件	版本号只升不降
0x1001	回滚成功事件码 (last_error 记录)	正常降级事件, 非故障
0x1002 BACKUP failed	备份当前 RUN 到 img_lib 失败	检查外部 Flash; 重推
0x1003 APP erase failed	擦除 RUN 失败	复位后由 BACKUP 自愈; 重推
0x1004 APP write failed	解密写入失败	复位后由 BACKUP 自愈; 重推
0x1005 APP vector invalid	写入后向量表 (SP/PC) 非法	复位后由 BACKUP 自愈; 重推
TCP PUSH FAILED	设备 :9020 不可达	检查 ETH 链路/设备是否运行 (复位后重试)

12.2 常见故障排查步骤

12.2.1 症状 1: 推送显示成功但固件没变

- **第一检查:** build 号未递增 → 被防重放拒绝。
- **确认:** 读设备 PARAM last_build_no (0x080E0014) 对比推送 build。
- **辅助:** 看 BOOT 阶段是否有三短音 (校验失败提示)。

12.2.2 症状 2: 设备不响应 :9020

- **判断:** ping 192.168.10.10——ICMP 通但 :9020 不通 → 设备半死 (TIM7 停)。
- **解决:** 手动复位设备 → 系统完整运行 → 重推。

12.2.3 症状 3: 升级后启动黑屏/异常

- **机制:** 启动确认未完成 (PENDING 超限) → 自动回滚。
- **确认:** 检查 PARAM boot_state (1=NORMAL, 2=PENDING) ; 观察是否回滚。

12.2.4 症状 4: 断点续传失败

- **原因:** 会话槽被 BOOT 提交清理 (正常) ; 或版本不一致 (不同包不续传)。
- **解决:** 重新 BEGIN 完整下载。

12.3 诊断三板斧

- ① **读参数区**：DAP 读 PARAM (0x080E0000 起)：boot_state/count/error/build_no 一目了然。
- ② **看日志**：COM5 串口日志：OTA Agent 打印 begin/data/end/entering BOOT；[SEC] 拒绝原因。
- ③ **验 Flash**：DAP 读 Flash 向量表比对 bin (0x08010000 前 16B) 确认固件是否真被替换。

第 13 章 FAQ

问题	回答
升级时业务会中断吗？	下载阶段不中断（APP 持续运行）；仅复位切换瞬间中断（<1s 级）。
升级断电会变砖吗？	不会。4 阶段断电注入全部实测恢复（续传/重应用/回滚/自愈）。
固件包能跨设备刷吗？	不能。AES 密钥由芯片 UID 派生，包与设备绑定。
旧固件能重刷吗？	不能。build 防重放 + version 防回滚双防线。
APP 崩溃了还能升级吗？	能。YMODEM 救援路由 BOOT 直接接收（不依赖 APP）。
升级后新固件有问题怎么办？	PENDING 窗口内自动回滚（count 到 3）；BACKUP 自愈。
如何确认当前固件版本？	主界面固件卡显示 FW v213 b9181；或读 PARAM last_build_no。
多次失败会锁死吗？	回滚超限进入 RECOVERY 恢复模式，强制重刷即可解锁。

第 14 章 总结与展望

D00 OTA 系统以“三端闭环 + 双区布局 + 外部回滚源 + 端到端安全 + 多通道传输 + 全链路自愈”构建了工业级固件升级的完整范式。其设计可迁移到任何带内部/外部 Flash 的 MCU 平台。

- **展望**：CAN 通道落地、RDP 发布前实机验证、N-2 级回滚、批量集群管理。
- **工程化**：把“断电注入 4 阶段”固化为 CI 回归项，每次发版自动跑。

附录 A 关键数据结构（源码级）

A.1 ota_header_t (32B)

```
typedef struct {
    uint32_t magic;           /* 0x4F5441FE: OTA 包标识 */
    uint32_t version;         /* 固件版本: 防回滚 (只升不降) */
    uint32_t firmware_size;   /* 原始固件大小 (明文): 容量校验 ≤832KB */
    uint8_t aes_iv[12];       /* AES-CTR 初始向量 (+4B 零=16B) */
    uint32_t chip_id;         /* 目标芯片 ID: 防跨芯片 */
    uint32_t build_no;        /* 构建号: 防重放 (严格递增) */
} ota_header_t; /* pack(1), 32B; 包总长 = 32 + firmware_size + 64 */
```

A.2 boot_param_t (PARAM 双份)

```
typedef struct {
    uint32_t magic;           /* 0x50524D54 ('PRMT') 参数区魔数 */
    uint32_t boot_state;      /* 1=NORMAL 2=PENDING 3=RECOVERY 4=UPGRADE */
    uint32_t boot_count;      /* 启动计数 (PENDING 窗口计数) */
    uint32_t rollback_count;  /* 回滚计数 (超限进 RECOVERY) */
    uint32_t last_error;      /* 上次升级错误码 (0x1001-0x1005/SEC err) */
    uint32_t last_build_no;    /* 已接受最大构建号 (防重放基准) */
    uint32_t crc32;           /* 覆盖 magic..last_build_no */
} boot_param_t; /* pack(1) 32B; slot0@0x080E0000 slot1@0x080E0400 (间距 1KB) */
```

A.3 OTA 会话槽 (断点续传)

```
struct { magic 0x4F54414D; version; total; received; crc32; }
(PARAM 区 0x080E2000 起; BOOT 升级提交后失效)
```

A.4 TCP 帧

帧 = magic(0x5A) | cmd | len(2B BE) | payload | crc8 (poly 0x07, 初值 0, 覆盖 cmd+len+payload)

cmd: 0x01 BEGIN / 0x02 DATA / 0x03 END / 0x04 STATUS / 0x05 RESET; ACK 帧 cmd=0x80

BEGIN 载荷 = version(4B BE) + size(4B BE); DATA 载荷 = offset(4B BE) + 240B 块

(帧总长上限 249B; 多字节一律大端, 与 HOSTLINK 的小端相反, 勿混用)

附录 B 术语表

术语	说明
OTA	Over-The-Air, 空中/远程固件升级
BOOT	启动引导器 (0x08000000, 64KB) ——校验/升级/回滚执行者
RUN	APP 运行区 (0x08010000, 832KB)
PARAM	参数区 (0x080E0000, 128KB) ——状态/防重放/续传

术语	说明
ota_dl	外部 Flash 下载暂存槽 (2MB, 单槽 1MB)
img_lib	外部 Flash 回滚源槽 (头 4KB + 数据 832KB)
PENDING	升级待确认状态 (新固件首启确认后转 NORMAL)
NORMAL	正常运行态
RECOVERY	恢复模式 (回滚超限; 无有效包自动归一 NORMAL 回旧固件, 不砖机)
UPGRADE	升级模式 (执行升级流程)
HOSTLINK	上位机串口链路协议 (UART1 921600)
YMODEM	传统串口文件传输协议 (BOOT 救援路径, 115200)
AES-CTR	Advanced Encryption Standard, 计数器模式 (流式加解密)
ECDSA	椭圆曲线数字签名算法 (P-256)
SHA-256	安全哈希算法 (32B 指纹)
UID	芯片唯一 ID (AES 密钥派生输入)
RDP	Read-out Protection (读保护, 防固件提取)
IWDG	独立看门狗 (硬件, LSI 时钟)
WDOG	任务级看门狗 (软件, SysMon)
BKP	备份寄存器 (RTC 域, 升级标志 0x5A5A 等)

附录 C 参考文档与源码索引

资料	位置
OTA 架构全貌	APP/APP/Doc/OTA_ARCHITECTURE.md
工程复盘日志 (12.x OTA 章节)	APP/APP/Doc/ENGINEERING_LOG.md
上位机用户指南	docs/OTA_TOOL_USER_GUIDE.md
BOOT 升级主流程	BOOT/BOOT/BootTestApp/boot_app.c
安全校验	BOOT/BOOT/BootTestServices/security.c
参数区管理	BOOT/BOOT/BootTestServices/boot_param.c/h
回滚备份	BOOT/BOOT/BootTestServices/ota_backup.c

资料	位置
外部 Flash 驱动	BOOT/BOOT/BootServices/esp_flash.c
APP 下载核心	APP/APP/Application/ota_agent.c
TCP 通道	APP/APP/Application/ota_tcp_svc.c
CAN 通道	APP/APP/Application/ota_can_svc.c + Config/can_proto.h
传输注册表	APP/APP/Application/ota_transport.h / ota_mgr.c
命令行推送	APP/APP/Script/ota_tcp_cli.py